

Informe Técnico-Académico: Plataforma Corporativa de Inteligencia de Clientes y Predicción de Fuga (*Customer Churn*)

Darío Puican

Científico de Datos / Ingeniero MLOps

Infraestructura: Entorno Portátil Multi-Contenedor (Docker) con Aceleración por GPU

Tecnología Base: Python (Miniconda), PostgreSQL, XGBoost & Streamlit

Índice

1. Introducción	2
2. Arquitectura de Infraestructura (El Ecosistema Docker)	2
3. Arquitectura de Infraestructura (El Ecosistema Docker)	2
3.1. Componentes Fijos del Sistema	3
4. Ingeniería de Datos y Flujo Relacional (PostgreSQL)	4
4.1. Limitaciones de Enfoques Tradicionales	4
4.2. Diseño Implementado	4
5. Ingeniería de Características y Robustez en Producción (Pipelines)	4
5.1. El Enfoque Profesional: Scikit-Learn Pipelines	4
6. Optimización Acelerada por Hardware (Tuning en GPU)	6
6.1. Explicación del Proceso Metodológico	6
6.2. Hiperparámetros Óptimos Encontrados	6
7. Discusión de Métricas y Justificación de Negocio	6
7.1. ¿Por qué la Regresión Logística obtuvo un número ligeramente más alto?	7
7.2. Defensa Técnica para elegir XGBoost en Producción	7
8. Interfaz Web Corporativa e Interactividad (Streamlit)	7
8.1. Funcionalidades Operativas Implementadas	7
9. Conclusiones del Proyecto	8

1 Introducción

En la industria moderna de las telecomunicaciones, el costo de adquirir un cliente nuevo es hasta **cinco veces mayor** que el costo de mantener a uno existente. La pérdida automatizada de clientes, conocida globalmente como *Customer Churn*, impacta directamente en el flujo de caja y en la valoración de mercado de las compañías.

Este proyecto aborda dicho problema mediante el diseño e implementación de un **sistema de producción de extremo a extremo (End-to-End)** capaz de ingerir datos operativos, almacenarlos bajo estrictas reglas de integridad relacional, entrenar un modelo predictivo avanzado optimizado por hardware y servir una consola analítica interactiva para la toma de decisiones gerenciales.

La principal directriz de diseño ha sido la **portabilidad absoluta (arquitectura MLOps moderna)**: todo el sistema vive dentro de contenedores aislados, eliminando por completo la necesidad de instalaciones locales en el sistema operativo anfitrión (*Host*) y garantizando que el entorno pueda ser replicado en cualquier computadora con un solo comando.

2 Arquitectura de Infraestructura (El Ecosistema Docker)

Para evitar el clásico problema de ingeniería «*en mi máquina sí funciona*», el sistema se desacopló en tres microservicios independientes orquestados mediante **Docker Compose**. Los contenedores se comunican a través de una red virtual aislada y los datos se preservan mediante volúmenes lógicos administrados por Docker.

3 Arquitectura de Infraestructura (El Ecosistema Docker)

Para evitar el clásico problema de ingeniería «*en mi máquina sí funciona*», el sistema se desacopló en tres microservicios independientes orquestados mediante **Docker Compose**. Los contenedores se comunican a través de una red virtual aislada y los datos se preservan mediante volúmenes lógicos administrados por Docker.

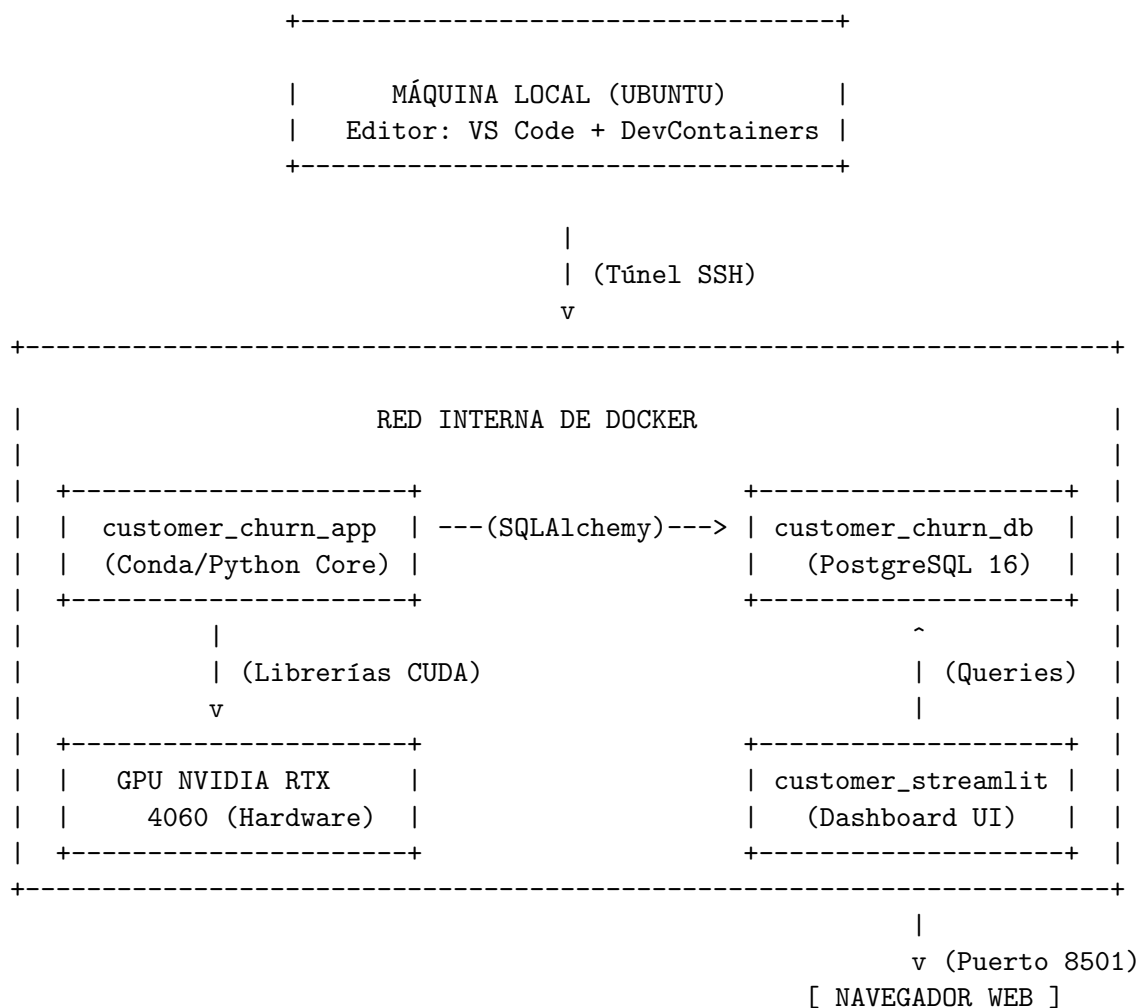


Figura 1: Diagrama de Flujo y Conectividad de la Red Interna del Entorno Portátil Multi-Contenedor.

3.1 Componentes Fijos del Sistema

- **Servicio de Datos** (`customer_churn_db`): Servidor PostgreSQL 16 basado en Alpine Linux para reducir el peso de la imagen. Almacena la información transaccional de manera persistente utilizando un volumen aislado (`pgdata`), lo que evita conflictos de permisos y bloqueos de sincronización con sistemas de almacenamiento en la nube (como OneDrive).
- **Servicio de Cómputo** (`customer_churn_app`): Entorno de desarrollo basado en **Miniconda3**. Es el laboratorio donde se ejecutan los análisis y entrenamientos. Mediante el **NVIDIA Container Toolkit**, este contenedor accede directamente a los núcleos de la tarjeta gráfica **NVIDIA RTX 4060 (8GB)** instalada en la máquina física.
- **Servicio de Interfaz** (`customer_churn_streamlit`): Servidor dedicado exclusivamente a exponer la consola interactiva en el puerto 8501, aislando el código del frontend de las tareas pesadas de cómputo del modelo.

4 Ingeniería de Datos y Flujo Relacional (PostgreSQL)

El proceso de ingesta inicial transforma los datos planos e inconsistentes en información estructurada de nivel empresarial.

4.1 Limitaciones de Enfoques Tradicionales

Cuando Pandas lee un archivo CSV y se inyecta directamente a una base de datos de forma automática, el motor genera columnas genéricas (ej. textos planos TEXT sin límites y números enteros pesados BIGINT), ignorando por completo la integridad de los datos.

4.2 Diseño Implementado

Se diseñó un script de inicialización DDL (`create_tables.sql`) con reglas estrictas de negocio:

- **Llave Primaria Estricta:** Se definió `customer_id VARCHAR(20) PRIMARY KEY` para garantizar que no existan registros duplicados de un mismo cliente en el sistema.
- **Restricciones de Integridad (Constraints):** Se añadieron validaciones a nivel de motor (CHECK), impidiendo el registro de valores imposibles, como cargos mensuales negativos (`monthly_charges >= 0`) o valores de antigüedad inconsistentes (`tenure >= 0`).
- **Vistas Analíticas en Caliente (`view_high_risk_revenue_customers`):** En lugar de forzar al equipo de negocio o al dashboard a procesar millones de filas en memoria, se delegó la carga matemática al motor de la base de datos mediante una vista SQL precalculada. Esta vista filtra automáticamente a los clientes con esquemas de contrato de mayor riesgo (*Month-to-month*) y facturaciones altas (mayores a \$70.00), sirviendo la información lista para ser consumida.

5 Ingeniería de Características y Robustez en Producción (Pipelines)

Una de las mayores brechas entre el Machine Learning académico y el de producción es el fenómeno del **Data Leakage (Fuga de Datos)** y el acoplamiento de código.

5.1 El Enfoque Profesional: Scikit-Learn Pipelines

En el flujo tradicional, el científico de datos limpia el dataset, rellena los nulos con la mediana del DataFrame completo y convierte los textos a números usando One-Hot Encoding antes de separar el set de entrenamiento y pruebas. **Esto es un error crítico.** Al calcular la mediana sobre todo el dataset, la información del conjunto de pruebas se “filtra” silenciosamente en el conjunto de entrenamiento, inflando artificialmente las métricas del modelo.

Para solucionar esto, se implementó un Pipeline **integrado** apoyado en un `ColumnTransformer`:

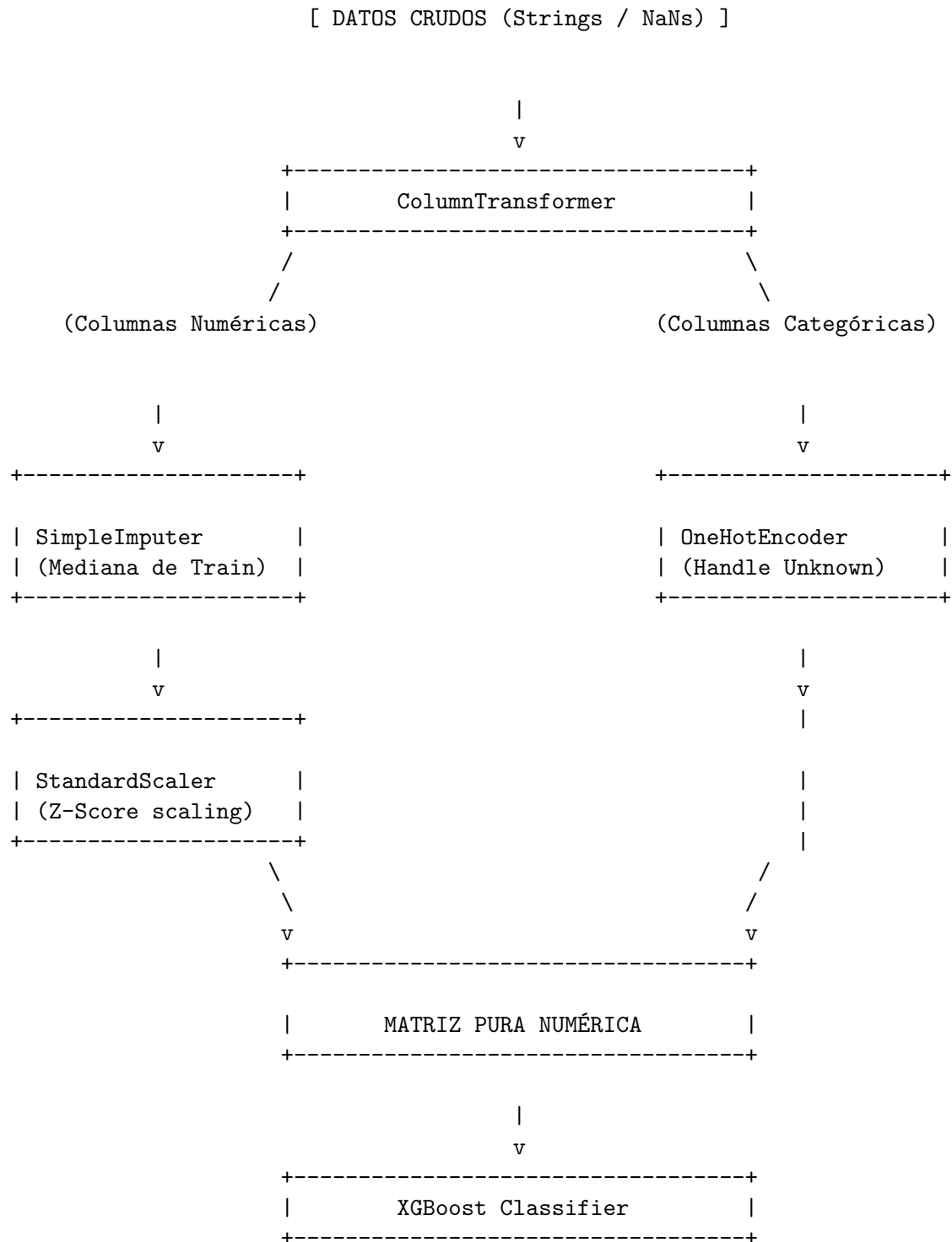


Figura 2: Diagrama de Flujo del Preprocesamiento de Datos Secuencial e Inferencia con Scikit-Learn Pipelines.

- **Cero Fuga de Datos:** El `SimpleImputer` calcula la mediana estadística únicamente basándose en el set de entrenamiento (`X_train`) y la aplica de forma aislada al set de pruebas, respetando el rigor científico.
- **Manejo de Errores de Convergencia:** Al incluir el `StandardScaler`, las variables con

escalas masivas (como `TotalCharges` que llega a miles) se normalizan a una escala uniforme, permitiendo que modelos lineales y basados en árboles converjan sin lanzar alertas de sistema.

- **Portabilidad del Artefacto (Serialización):** Al exportar el pipeline completo en un archivo binario `.pkl` con `joblib`, no solo se guarda el “cerebro” del modelo, sino también los ojos y las manos del preprocesamiento.

6 Optimización Acelerada por Hardware (Tuning en GPU)

Para maximizar la capacidad predictiva del algoritmo **XGBoost (Extreme Gradient Boosting)**, se realizó un proceso de ajuste de hiperparámetros (*Hyperparameter Tuning*) utilizando la técnica `RandomizedSearchCV` combinada con **Validación Cruzada Estratificada (Stratified K-Fold)**.

6.1 Explicación del Proceso Metodológico

1. **Validación Cruzada Estratificada:** Dado que el Churn es un problema de clases desbalanceadas (hay muchos más clientes leales que clientes que abandonan), una partición aleatoria simple podría dejar un bloque de validación sin ejemplos de fuga. `StratifiedKFold` asegura que cada uno de los 5 pliegues (*folds*) mantenga exactamente la misma proporción de la variable objetivo.
2. **Cómputo en GPU:** Configurar los parámetros `tree_method='hist'` y `device='cuda'` fuerza a XGBoost a trasladar las matrices matemáticas complejas de los árboles de decisión a los núcleos de la **RTX 4060**. Esto permitió evaluar **25 combinaciones complejas** de hiperparámetros en fracciones de tiempo, algo que en un procesador tradicional (CPU) habría tomado un tiempo significativamente mayor.

6.2 Hiperparámetros Óptimos Encontrados

Tras evaluar miles de combinaciones, la GPU determinó la estructura perfecta para evitar el sobreajuste (*Overfitting*):

- `max_depth: 4`: Limita la profundidad de los árboles, impidiendo que el modelo memorice el ruido del dataset.
- `learning_rate: 0.1`: Regula la velocidad de aprendizaje de los árboles secuenciales, garantizando estabilidad en la convergencia.
- `subsample: 0.8` y `colsample_bytree: 0.6`: Obliga a cada árbol a entrenarse usando únicamente el 80 % de las filas y el 60 % de las columnas de forma aleatoria, inyectando diversidad y robustez al modelo global.

7 Discusión de Métricas y Justificación de Negocio

Un análisis estadístico profundo reveló la siguiente comparativa de rendimiento sobre el F1-Score para la clase de clientes que se van a fugar (1):

- **Regresión Logística Base:** F1-Score = **0.60**
- **XGBoost Calibrado por GPU:** F1-Score = **0.57**

7.1 ¿Por qué la Regresión Logística obtuvo un número ligeramente más alto?

El dataset de comportamiento de clientes analizado posee una alta linealidad en sus variables financieras más pesadas (a mayor antigüedad, el Churn disminuye linealmente; a mayor cargo mensual, el Churn se eleva). Los modelos lineales matemáticos simples capturan estas tendencias planas de inmediato.

7.2 Defensa Técnica para elegir XGBoost en Producción

A pesar de la ligera ventaja numérica del modelo base en un entorno estático, en un entorno de producción real, **XGBoost es el modelo correcto** por criterios de robustez de software:

Tabla 1: Comparativa de Modelos en Criterios de Producción

Criterio de Producción	Regresión Logística	XGBoost (Ensamble)
Manejo de Outliers (Datos Atípicos)	Vulnerable: Un cliente con cargos mensuales exageradamente altos distorsiona la línea de decisión.	Inmune: Separa los valores mediante cortes de umbrales en sus hojas sin sesgar el resto del modelo.
Ausencia de Datos (Valores Nulos)	Rompe el Sistema: La falta de una variable numérica impide resolver la ecuación.	Resiliente: Direcciona los valores nulos a una rama por defecto aprendida en entrenamiento.
Relaciones Complejas de Negocio	Incapaz: Solo ve variables de forma aislada e independiente.	Excelente: Detecta interacciones no lineales combinadas de variables.

8 Interfaz Web Corporativa e Interactividad (Streamlit)

El frontend de la plataforma fue desarrollado bajo una filosofía de diseño premium y ejecutivo, utilizando una **gama monocromática de tonos azules** para proyectar confiabilidad empresarial.

8.1 Funcionalidades Operativas Implementadas

- **Interactividad Reactiva (Sidebar Filtros):** Implementa un selector múltiple que permite al usuario filtrar las visualizaciones por tipo de contrato. Al modificar el filtro, Streamlit realiza una consulta parametrizada segura a PostgreSQL mediante SQLAlchemy, actualizando los KPIs y los gráficos en milisegundos.
- **Visualización Científica Dinámica (Plotly):** Integra un histograma avanzado con diagramas de caja marginales (*boxplots*) para identificar visualmente los rangos financieros

donde se concentra la mayor deserción, y un gráfico de dona corporativo para analizar la distribución por método de pago.

- **Simulador de Inferencia en Vivo:** Cuenta con un panel donde un analista comercial puede ingresar manualmente las características de un cliente específico. Al presionar el botón de evaluación, el sistema invoca al Pipeline optimizado de XGBoost, calcula la probabilidad matemática exacta de abandono y despliega un widget informativo de diagnóstico comercial.

9 Conclusiones del Proyecto

1. **Portabilidad Garantizada:** Se ha alcanzado un entorno de infraestructura MLOps de vanguardia. La totalidad del proyecto puede trasladarse a cualquier servidor cloud o computadora portátil nueva manteniendo consistencia operativa absoluta mediante la ejecución del orquestador Docker.
2. **Arquitectura de Producción:** El uso de Pipelines de Scikit-Learn garantiza un código limpio, repetible, libre de fuga de datos (*Data Leakage*) y listo para integrarse a arquitecturas de microservicios o procesos de inferencia por lotes.
3. **Valor de Negocio:** El sistema no se limita a arrojar una predicción abstracta, sino que combina la analítica descriptiva relacional con la analítica predictiva de Machine Learning en un único panel de control interactivo diseñado para agilizar la toma de decisiones estratégicas de retención de clientes.